

Calidad de Software

Ingeniería de Software 4

Luz Mery Rios Alzate / Ingeniería de Sistemas

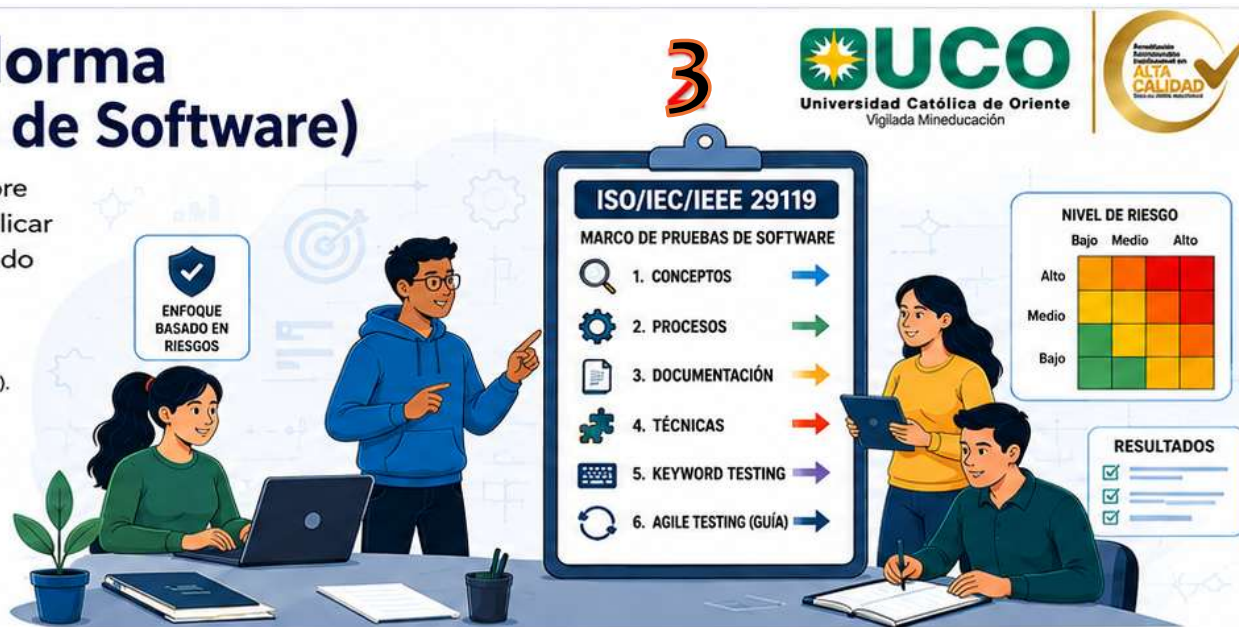
Estudio y Exposición de la Norma ISO/IEC/IEEE 29119 (Pruebas de Software)

Investigar, comprender y explicar la norma ISO/IEC/IEEE 29119 sobre Pruebas de Software, desarrollando la capacidad de entender y aplicar sus conceptos, procesos, técnicas, documentación y enfoque basado en riesgos, mediante una exposición práctica y didáctica.

Aspectos de la norma que pueden ser seleccionados:

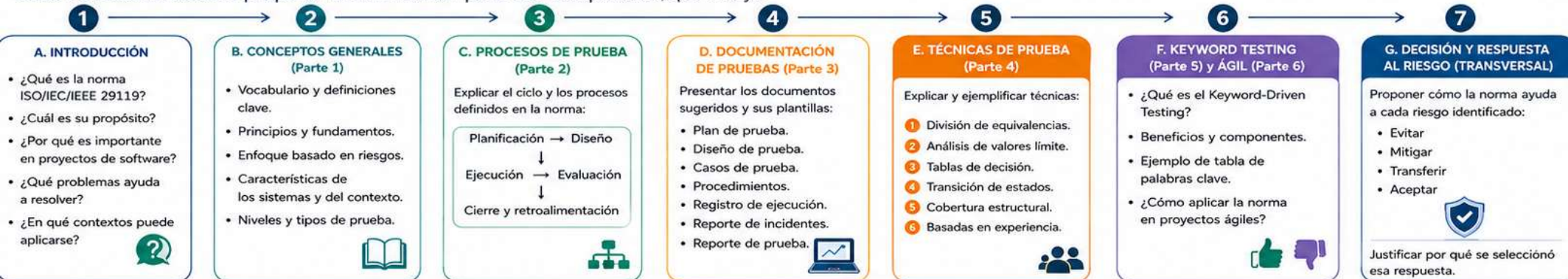
- ✓ Conceptos generales (Parte 1).
- ✓ Procesos de prueba (Parte 2).
- ✓ Documentación de pruebas (Parte 3).
- ✓ Técnicas de prueba (Parte 4).
- ✓ Keyword-Driven Testing (Parte 5).
- ✓ Pruebas en contexto ágil (Parte 6 - Guía).
- ✓ Enfoque basado en riesgos.
- ✓ Roles y responsabilidades.
- ✓ Niveles de prueba.
- ✓ Gestión de defectos.

Propósito de la actividad: No se trata solamente de explicar la norma, sino de **enseñarla** a los compañeros y demostrar cómo se utiliza en un proyecto de software mediante un ejemplo práctico.



Estudio y Exposición de la Norma ISO/IEC/IEEE 29119 (Pruebas de Software)

Cada estudiante deberá preparar un mini-taller para sus compañeros, que incluya:



Incluir en la exposición:

2 LIBROS relacionados con pruebas de software, calidad de software o gestión de pruebas.

2 ARTÍCULOS CIENTÍFICOS O TRABAJOS DE GRADO relacionados directamente con la norma 29119.

i Importante: las fuentes no deben limitarse a mencionarse en la bibliografía; durante la exposición deben explicar qué aporte hace cada fuente al tema estudiado.

Toma de Decisiones en Ingeniería de Software Basada en CMMI y la ISO

Analizar, comprender y aplicar modelos de referencia como CMMI y las normas ISO para tomar decisiones informadas, justificadas y basadas en evidencia en proyectos de software, considerando calidad, riesgo y mejora continua.

Aspectos clave que puedes explorar:

- ✓ Modelos de referencia: CMMI e ISO.
- ✓ Procesos y áreas de proceso.
- ✓ Niveles de madurez y capacidad.
- ✓ Datos e indicadores para la decisión.
- ✓ Evaluación de alternativas.
- ✓ Gestión de riesgos en la toma de decisiones.
- ✓ Calidad del producto y del proceso.
- ✓ Cumplimiento de estándares.
- ✓ Trazabilidad y evidencia.
- ✓ Costo – beneficio.
- ✓ Contexto organizacional.
- ✓ Mejora continua.



Propósito de la actividad: Desarrollar la capacidad de tomar decisiones en proyectos de software utilizando criterios y prácticas de CMMI y la ISO, mediante un caso práctico, análisis de alternativas y justificación documentada.

DECISIONES
INFORMADAS
=
MEJORES
RESULTADOS



CRITERIOS PARA LA TOMA DE DECISIONES

1. CONTEXTO Y OBJETIVOS
2. ALTERNATIVAS
3. EVIDENCIA Y DATOS
4. ANÁLISIS Y EVALUACIÓN
5. DECISIÓN
6. SEGUIMIENTO Y MEJORA

CMMI (Niveles de madurez)



ISO/IEC/IEEE

29110
(Very Small Entities)

Toma de Decisiones con CMMI y la ISO (Actividad en Equipo o Individual)

Cada estudiante deberá desarrollar un mini-taller o caso de análisis que incluya:



A. INTRODUCCIÓN

- ¿Cuál es la situación o problema de decisión?
- ¿Qué objetivos se quieren alcanzar?
- ¿Qué relación tiene con CMMI y la ISO?
- ¿Qué información es relevante para decidir?



B. MARCO DE REFERENCIA

- Conceptos clave de CMMI (niveles, áreas de proceso).
- Elementos relevantes de la ISO.
- Buenas prácticas relacionadas con la toma de decisiones.
- Cómo los modelos aportan al contexto del caso.



C. IDENTIFICACIÓN DE ALTERNATIVAS

- Definir al menos 2 o 3 alternativas de solución.
- Describir ventajas y desventajas de cada una.
- Considerar criterios como: calidad, costo, tiempo, riesgo, madurez de la organización.



D. ANÁLISIS Y EVALUACIÓN

- Aplicar un método de evaluación (matriz de decisión, AHP, ponderación, etc.).
- Usar datos e indicadores cuando sea posible.
- Relacionar la evaluación con prácticas de CMMI y la ISO.
- Mostrar la justificación del análisis.



E. DECISIÓN

- Seleccionar la alternativa más adecuada.
- Justificar la decisión tomada.
- Explicar cómo cumple con los principios de CMMI/ISO.
- Considerar impacto en el proyecto y en la organización.



F. PLAN DE IMPLEMENTACIÓN Y SEGUIMIENTO

- Definir acciones para implementar la decisión.
- Identificar responsables.
- Determinar indicadores de seguimiento.
- Proponer mejoras continuas.



G. CONCLUSIONES Y APRENDIZAJE

- ¿Qué se aprendió del proceso de decisión?
- ¿Cómo ayudan CMMI y la ISO a tomar mejores decisiones?
- ¿Qué otros factores se deben considerar en el futuro?



Incluir en la entrega:

2 REFERENCIAS BIBLIOGRÁFICAS
(1 sobre CMMI y 1 sobre ISO).

1 ARTÍCULO CIENTÍFICO O TRABAJO DE GRADO
relacionado con toma de decisiones, CMMI o la ISO.



Importante: las fuentes deben estar correctamente citadas (APA).
La decisión debe estar fundamentada en evidencia, no solo en opinión.

Modelos de Desarrollo de Software

Modelo	Cuándo usarlo correctamente	Por qué
Cascada	Usarlo cuando los requisitos están muy claros y estables desde el inicio, el alcance cambia poco y conviene seguir fases secuenciales con alta documentación y control.	Porque funciona mejor cuando el alcance es estable, hay poca incertidumbre y se valora la secuencia, documentación y control.
Incremental – Incrementos	Usarlo cuando el sistema puede dividirse en módulos o funcionalidades entregables de forma progresiva, permitiendo liberar valor temprano y construir el producto por partes.	Porque permite entregar valor por partes, priorizar funcionalidades y recibir retroalimentación sin esperar al producto completo.
Incremental – Iteraciones	Usarlo cuando los requisitos evolucionan, hay incertidumbre o se necesita mejorar el producto en ciclos sucesivos con aprendizaje y retroalimentación constante.	Porque facilita aprender, ajustar y refinar el producto en ciclos cuando los requisitos cambian o maduran con el tiempo.

A la Verdad, por la fe y la ciencia

Academy
on Rheumatism
**ALTA
CALIDAD**
Society of Rheumatology (SFER)
Society of Rheumatoid Arthritis (SFERA)

Vigilada Mineducación



Planner

Falta de definición de tareas



- ☐ Las tareas creadas **no tienen criterios de aceptación.**

Problemas de planificación



- ☐ Las tareas **no tienen fechas de inicio.**
- ☐ Las tareas **no tienen fechas de vencimiento.**

Problemas de seguimiento



- ☐ Existen **tareas sin asignación.**
- ☐ Algunas tareas **no registran progreso.**
- ☐ Estudiantes sin tareas

⚠ Impacto:

La ausencia de criterios, responsables y fechas dificulta la planificación, el seguimiento y la medición del avance de las actividades.

A la Verdad, por la fe y la ciencia

Parcial

☐ **Punto 1:** Teoría. Valor: 25%

☐ **Punto 2:** Trabajo: ceiba-dev-fest. Valor: 25%

- speakers:
<https://www.ceiba.com.co/enterate/eventos/ceiba-dev-fest/2026/>
- Agenda:
<https://www.ceiba.com.co/enterate/eventos/ceiba-dev-fest/2026/>
- Inscripción:
<https://docs.google.com/forms/d/e/1FAIpQLSfWwAT9JbDdTOHIQDdFwDjuZsc9bQKjGKM9s5go6bpl0MdoHQ/viewform>

☐ **Punto 3:** Entrega de artefactos del proyecto y Plan de pruebas y casos de prueba. Valor 50%



A la Verdad, por la fe y la ciencia

La tecnología
crece,
pero el criterio
humano
multiplica su valor.

Ceiba DevFest 2026

DEL EVENTO A LA PROFESIÓN

Escuchar · Comprender · Analizar · Cuestionar · Relacionar · Aplicar

Ideas hoy,
mejores ingenieros
mañana

Actividad de reflexión profesional

Convierte tu experiencia en aprendizaje para tu futuro como ingeniero de software



Importante

La actividad **no** consiste en copiar o transcribir el contenido de las conferencias.

Se busca evidenciar la capacidad de **escuchar, comprender, analizar, cuestionar, relacionar y aplicar** lo aprendido.



Propósito

Participar en el Ceiba DevFest 2026 y transformar la experiencia en una reflexión sobre el ejercicio profesional de la ingeniería de software.



3. Evidencias de participación

Adjunte las evidencias que permitan demostrar su participación en el evento (por ejemplo, captura de pantalla, certificado o evidencia suministrada por la organización).



Información general de la conferencia

Título de la conferencia: _____

Conferencista(s): _____

Empresa / cargo (si se presenta): _____

Hora de la conferencia: _____



¿Qué aprendí?

Identifique la idea o concepto que considera más importante de la conferencia y explíquelo con sus propias palabras.



¿Qué me hizo pensar?

Identifique una idea que haya generado en usted una reflexión, duda, acuerdo o desacuerdo. Explique por qué.



¿Cómo se relaciona con mi carrera?

Relacione lo aprendido con una asignatura, concepto, competencia o experiencia de su formación como ingeniero de software.



¿Cómo lo aplicaría?

Imagine que ya está trabajando como profesional. Describa una situación real en la que podría aplicar lo aprendido.



¿Qué riesgo tendría?

Explique qué podría ocurrir si esta tecnología, metodología o idea se utiliza de manera incorrecta o sin criterio profesional.



Una pregunta que me queda

Escriba una pregunta que le haya surgido después de escuchar la conferencia.



Evidencia de aprendizaje

Escriba una frase, concepto, ejemplo o situación de la conferencia que haya llamado especialmente su atención y explique por qué.

La tecnología crece, pero el criterio humano multiplica su valor.

Ceiba DevFest 2026

DEL EVENTO A LA PROFESIÓN

Escuchar · Comprender · Analizar · Cuestionar · Relacionar · Aplicar

Ideas hoy, mejores ingenieros mañana

Actividad de reflexión profesional

Convierte tu experiencia en aprendizaje para tu futuro como ingeniero de software



☐ **Comparación de las cuatro conferencias**

- ✓ **¿Qué tienen en común?** Identifique dos ideas que aparezcan de manera transversal en las cuatro conferencias.
- ✓ **2. ¿En qué se contradicen o presentan perspectivas diferentes?:** Seleccione dos ideas o enfoques y explique si existen diferencias entre los conferencistas.
- ✓ **3. ¿Cuál considero más relevante para mi futuro profesional?:** Seleccione una de las cuatro conferencias y justifique su elección.
- ✓ **4. ¿Qué conocimiento podría aplicar desde ahora?** Explique qué podría aplicar desde su etapa como estudiante y no solamente cuando tenga un empleo.

☐ **Reto profesional. “Si mañana comenzara a trabajar como ingeniero de software...”** A partir de las cuatro conferencias seleccionadas, explique cómo cambiaría su forma de trabajar.

- ✓ Una práctica que comenzaría a realizar
- ✓ Una práctica que dejaría de realizar
- ✓ Una competencia que debería fortalecer
- ✓ Una herramienta o tecnología que investigaría

☐ **Reflexión final – Mi futuro como ingeniero de Sistemas:** Después de asistir al Ceiba DevFest 2026, responda de manera argumentada:

- ✓ **¿Qué cambió en mi forma de entender el trabajo de un ingeniero de Sistemas?**
- ✓ **¿Qué tareas de un ingeniero de software podrían ser automatizadas por IA y cuáles deberían seguir dependiendo del criterio humano?**
- ✓ **¿Cuál de las conferencias tuvo mayor impacto en mi formación y por qué?**

☐ Después de asistir al Ceiba DevFest 2026, ¿qué significa para usted ser un buen ingeniero de software en una época en la que la inteligencia artificial puede generar código, automatizar tareas y apoyar muchas de las actividades tradicionales del desarrollo de software?

Bintec



Vive Bintec este 13, 14 y 15 de octubre

Banca, Innovación, Tecnología

Sé parte de las conversaciones, experiencias e innovaciones que definirán la próxima era de la banca.

Vívelo en Bogotá, Medellín o virtual.



A la Verdad, por la fe y la ciencia



Técnicas de Validación

Repaso



1. no implica la ejecución de código
2. Utiliza métodos como revisiones, recorridos, inspecciones y verificación documental,
3. verifica si el software confirma una especificación
4. encuentra los errores al principio del ciclo de desarrollo.
5. la realiza el equipo de control de calidad.

1. implica la ejecución de código.
2. Utiliza métodos como caja negra y caja blanca y pruebas no funcionales
3. verifica si el software cumple con los requisitos y expectativas.
4. encuentra los errores que la verificación no puede detectar.
5. se realiza mediante la participación del equipo de pruebas con el equipo de control de calidad.

Pruebas de Software

Beizer (BEI90,pag. 1) lo plantea de mejor manera diciendo:

existe un mito que dice que, si fuéramos buenos programando, no habría errores que buscar.

Elemento crítico para la garantía de la calidad del software y representa una revisión final de las especificaciones del diseño y codificación .

A la Verdad. por la fe y la ciencia

Repaso

Los objetivos de las pruebas

Es un proceso de ejecución de un programa con la intención de descubrir un error.

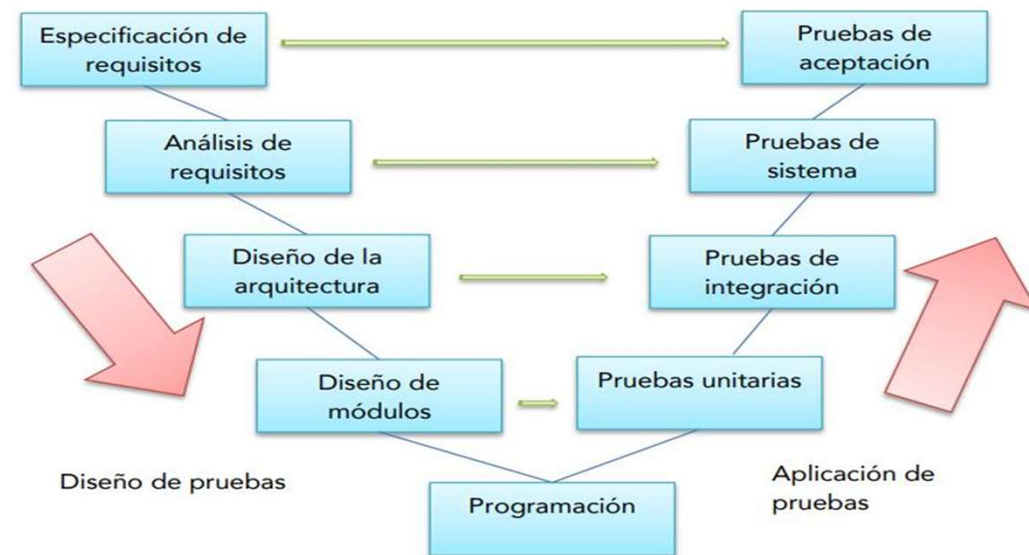


Casos de Prueba

Un buen caso de prueba es aquel que tiene una alta probabilidad de mostrar un error no descubierto hasta entonces.

Una prueba tiene éxito si descubre un error no detectado hasta entonces.

Ciclo en V



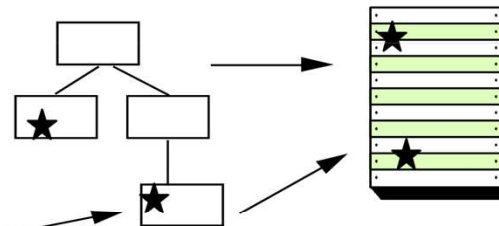
A la Verdad, por la fe y la ciencia

Relación entre Error, Defecto y Fallo

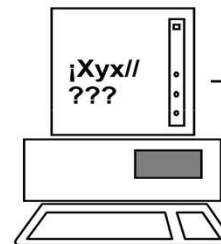
Error
Equivocación
del programador



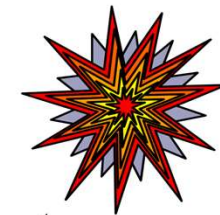
Sistema de gestión de aeropuerto



Defecto (calidad)



S. Aproximacion



Accidente
(seguridad)

<https://www.youtube.com/watch?v=aCxJ4Qo3bRg>

Por la Verdad, por la fe y la ciencia

Funcionales vs. No Funcionales

Tipo	¿Qué evalúa?	Ejemplos
Funcionales	El "Qué" hace el sistema.	Pruebas de humo, regresión, pruebas de API.
No Funcionales	El "Cómo" lo hace.	Rendimiento, seguridad, usabilidad, carga.

Pruebas Funcionales: Se centran en los requisitos del negocio.

- ☐ Pruebas de Humo (Smoke Testing): Pruebas rápidas para ver si las funciones básicas sirven (si la app "no echa humo" al encenderla).
- ☐ Pruebas de Regresión: Aseguran que un cambio nuevo no haya roto algo que ya funcionaba.

Pruebas No Funcionales: Se centran en la experiencia y calidad técnica.

- ☐ Pruebas de Rendimiento (Performance): Evalúan velocidad y estabilidad.
- ☐ Pruebas de Usabilidad: Evalúan qué tan intuitivo es el software.

Técnicas de Prueba

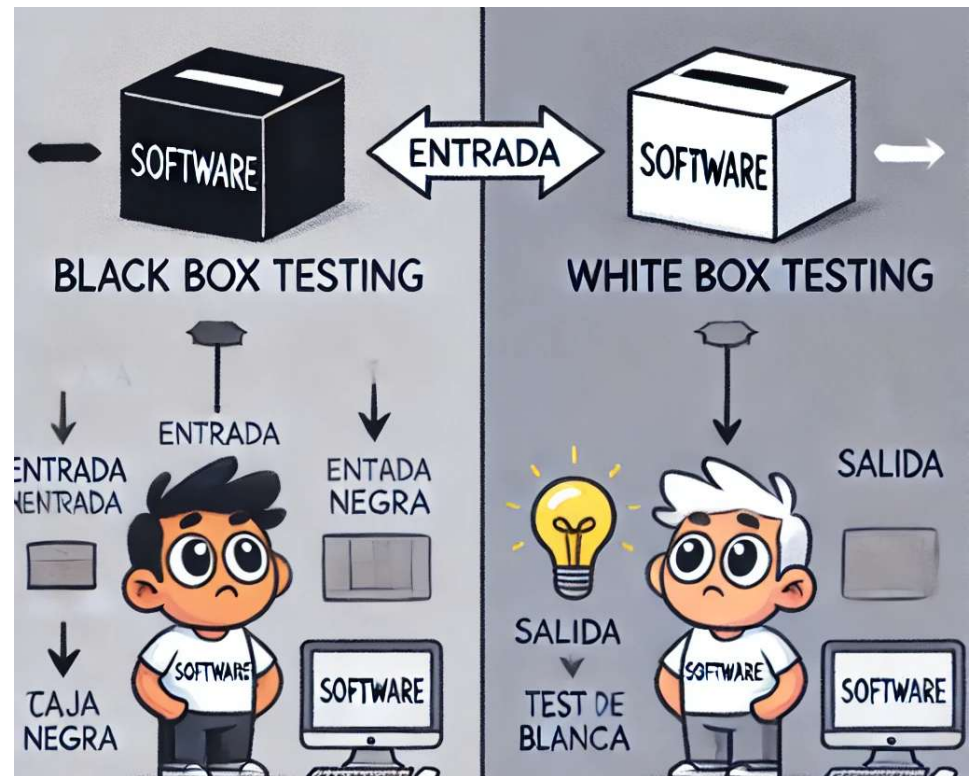
De forma general, existen **dos enfoques** diferentes:

Caja Negra (Funcional)

- ✓ Los casos de prueba se basan sólo en el comportamiento de entrada/salida

Caja Blanca (Estructural)

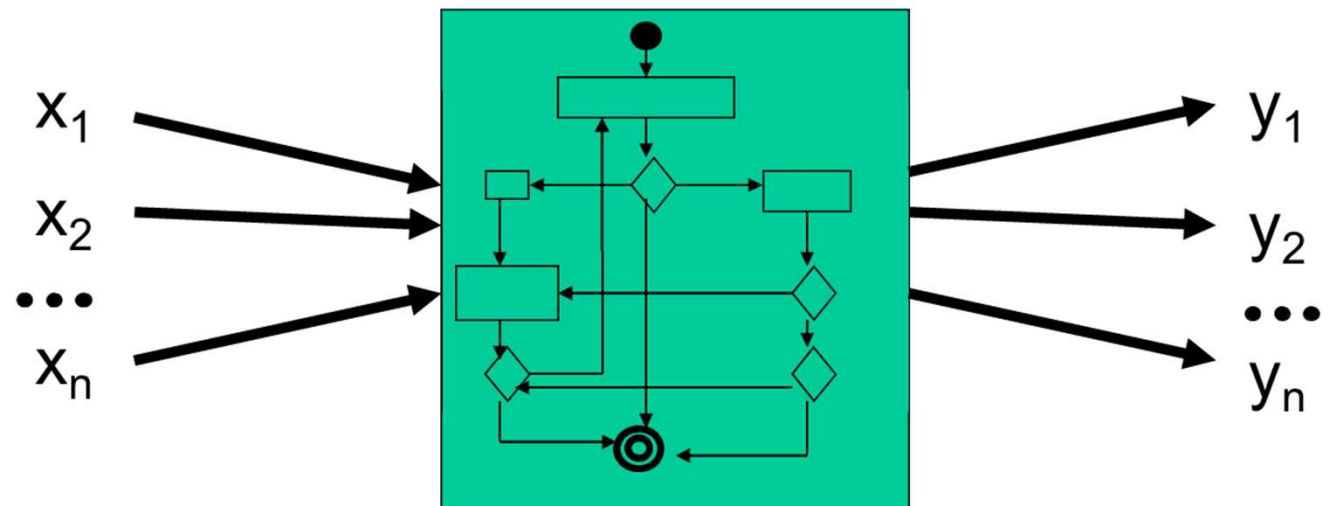
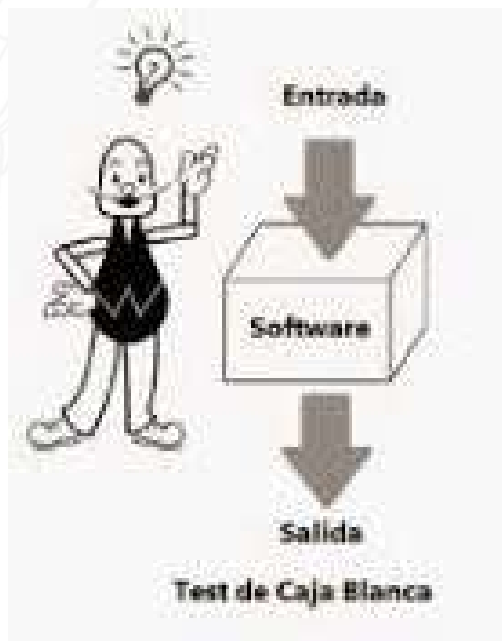
- ✓ Basadas en información sobre cómo el software ha sido diseñado o codificado



A la Verdad, por la fe y la ciencia

Prueba de Caja Blanca

De alguna manera, me interesa conocer la **cobertura** alcanzada por mis casos de prueba dentro de la unidad de prueba



A la Verdad, por la fe y la ciencia

Criterios de Cobertura:



Cobertura de sentencias. Que cada sentencia se ejecute al menos una vez.



Cobertura de decisiones. Que cada decisión tenga, por lo menos una vez, un resultado verdadero y, al menos una vez, uno falso.



Cobertura de condiciones. Que cada condición de cada decisión adopte el valor verdadero al menos una vez y el falso al menos una vez.

A la Verdad, por la fe y la ciencia

Prueba de Caja Blanca

- ☐ Prueba de camino Básico
- ☐ Prueba de la Estructura de Control.
- ☐ Prueba de condición
- ☐ Prueba de bucles



A la Verdad, por la ciencia

Pruebas de Caja Blanca. Camino Básico

❑ Propuesta por Tom McCabe (1976)

❑ **Objetivo:** Definir un conjunto básico de caminos de ejecución.

❑ **Pasos:**

A partir del diseño o del código fuente, dibujar el grafo de flujo asociado



Se calcula la complejidad ciclomática del grafo



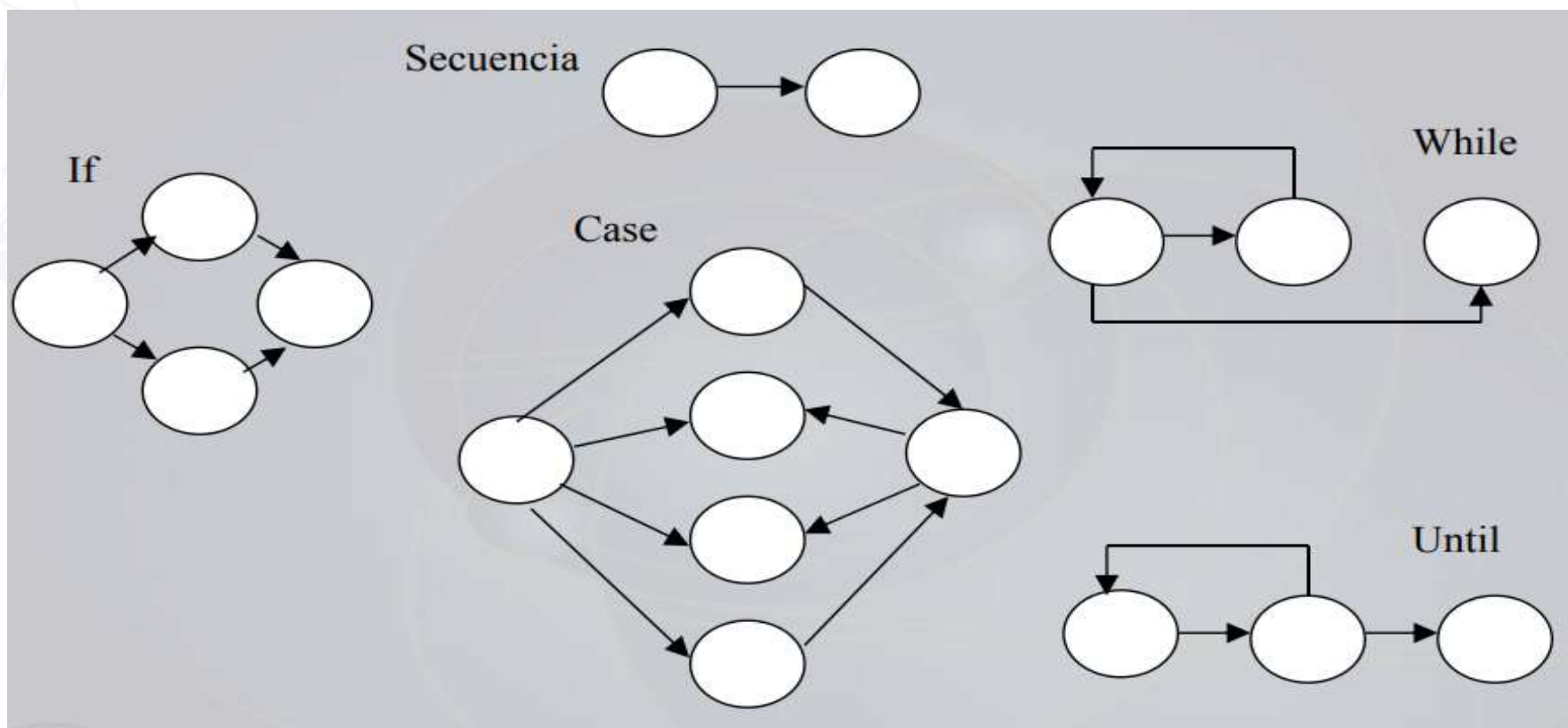
Se determina un conjunto básico de caminos independientes



Se preparan los casos de prueba que obliguen a la ejecución de cada camino del conjunto básico

Pruebas de Caja Blanca. Camino Básico

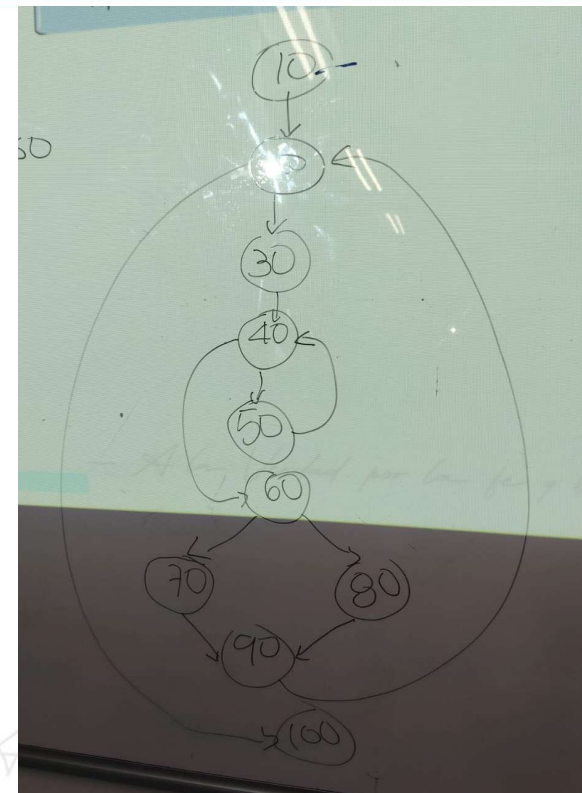
A partir del diseño o del código fuente, dibujar el grafo de flujo asociado



At the Verdad, por la fe y la ciencia

Pruebas de Caja Blanca. Camino Básico

A partir del diseño o del código fuente, dibujar el grafo de flujo asociado



y la ciencia

```

1  #include<iostream>
2  using namespace std;
3  int main() {
4      int n,m,c1,c2,suma, promedio,calf,ca=0,cna=0,promG=0;
5      cout<<"Introduce el numero de alumnos: ? ";
6      cin>>m;
7      cout<<"Introduce el numero de calificaciones: ? ";
8      cin>>n;
9      c1 = 1;
10     while (c1<=m)
11     {
12         suma=0;
13         cout<<"\n\nNo. Alumno "<<c1<<endl;
14         for(c2=1; c2<=n; c2++)
15         {
16             cout<<"\nCalificacion "<<c2<<": ? ";
17             cin>>calf;
18             suma +=calf;
19         }
20         promedio=suma/n;
21         promG +=promedio;
22         cout<<"\nEl promedio es "<<promedio;
23         if (promedio >=70)
24         {
25             ca++;
26             cout<<" Aprobado"<<endl;
27         }
28         else {
29             cna++;
30             cout<<" No aprobado"<<endl;
31         }
32         c1++;
33     }
34 }
  
```

Ejercicio último dígito par y cero

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int a, b, c;
6      cout << "Ingrese los tres lados del triangulo: ";
7      cin >> a >> b >> c;
8
9      if (a + b > c && a + c > b && b + c > a) {
10         if (a == b && b == c) {
11             cout << "Triangulo Equilatero" << endl;
12         } else if (a == b || b == c || a == c) {
13             cout << "Triangulo Isosceles" << endl;
14         } else {
15             cout << "Triangulo Escaleno" << endl;
16         }
17     } else {
18         cout << "No forma un triangulo valido" << endl;
19     }
20
21     return 0;
22 }
```

Verdad, por la fe y la ciencia

Ejercicio último dígito impar

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int edad;
6      bool tieneCupon;
7      double precioBase = 100.0, precioFinal;
8
9      cout << "Ingrese la edad: ";
10     cin >> edad;
11     cout << "Tiene cupon? (1: Si, 0: No): ";
12     cin >> tieneCupon;
13
14     if (edad < 12 || edad >= 65) {
15         precioFinal = precioBase * 0.5; // 50% de descuento
16     } else if (edad >= 12 && tieneCupon) {
17         precioFinal = precioBase * 0.8; // 20% de descuento
18     } else {
19         precioFinal = precioBase;
20     }
21
22     cout << "El precio final es: $" << precioFinal << endl;
23     return 0;
24 }
```

Verdad, por la fe y la ciencia

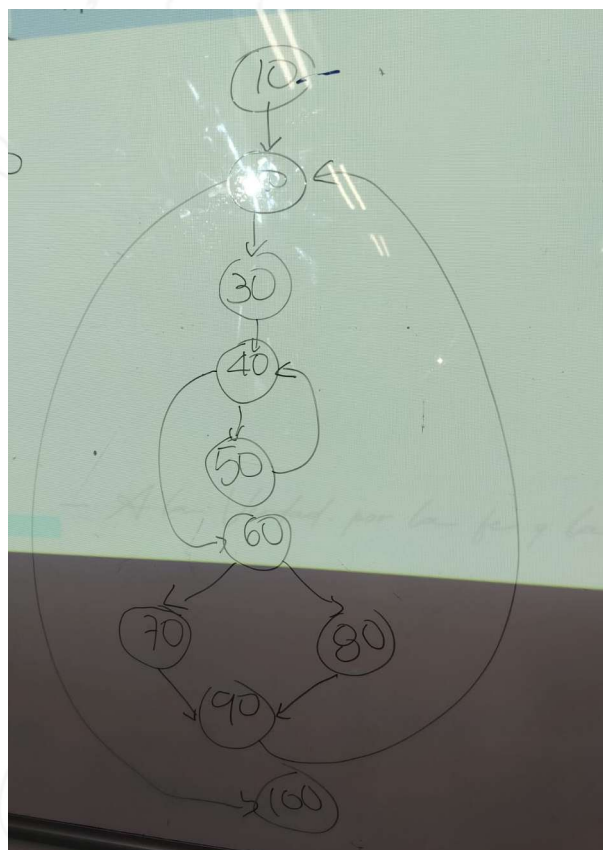
Pruebas de Caja Blanca. Camino Básico

Se calcula la complejidad ciclomática del grafo

Complejidad ciclomática de un grafo de flujo
 $V(G)$ establece el número de caminos
independientes.

Definición 1: Sean a : número de arcos y n : número de nodos; entonces la complejidad ciclomática o número ciclomático se define como:

$$v(G) = a - n + 2$$



A la Verdad, por la fe y la ciencia

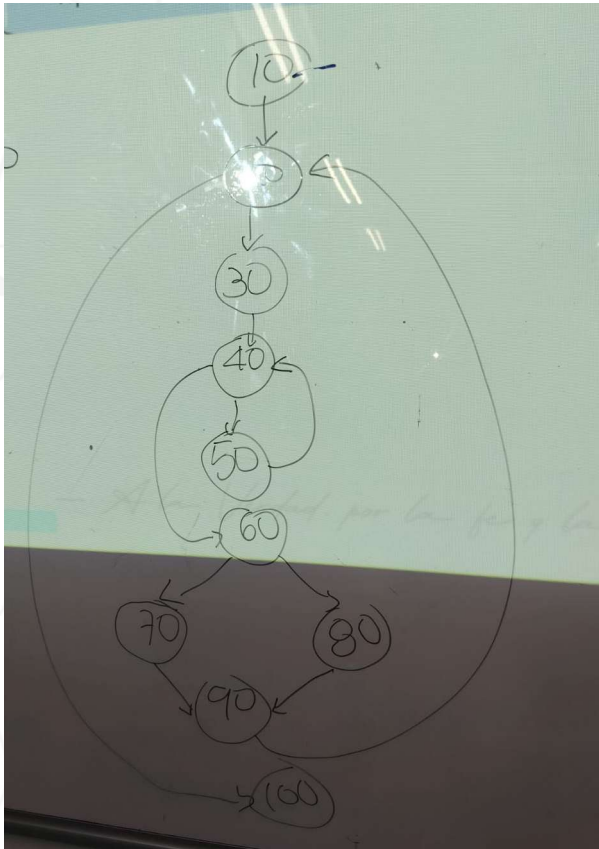
Pruebas de Caja Blanca.

Camino Básico

Se calcula la complejidad ciclomática del grafo

Definición 2: Sea nps el número de nodos con predicado simple (es decir, nodos de los que parten dos caminos); entonces la complejidad ciclomática o número ciclomático se define como:

$$v(G) = 1 + nps$$



A la Verdad, por la fe y la ciencia

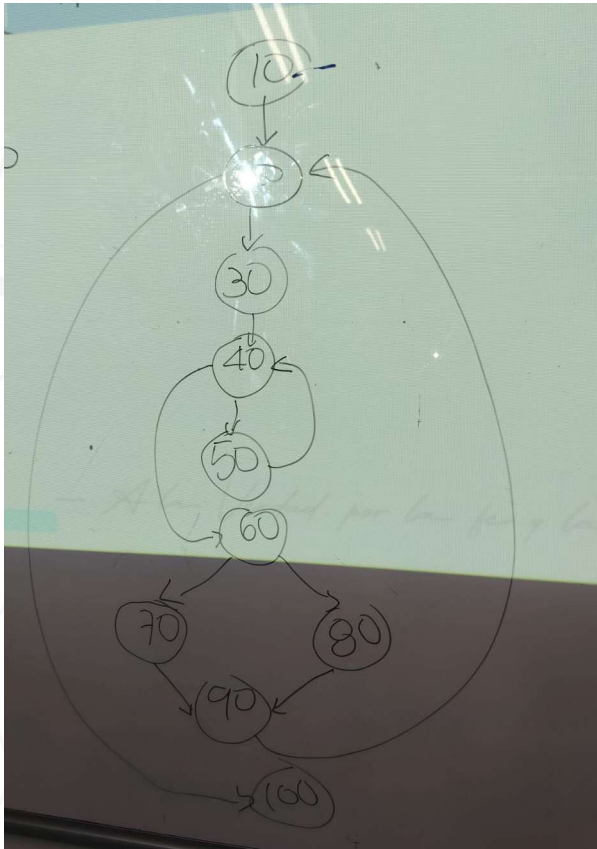
Pruebas de Caja Blanca.

Camino Básico

Se calcula la complejidad ciclomática del grafo

Definición 3: Sea rr el número de regiones rodeadas completamente por arcos del grafo; entonces la complejidad ciclomática o número ciclomático se define como:

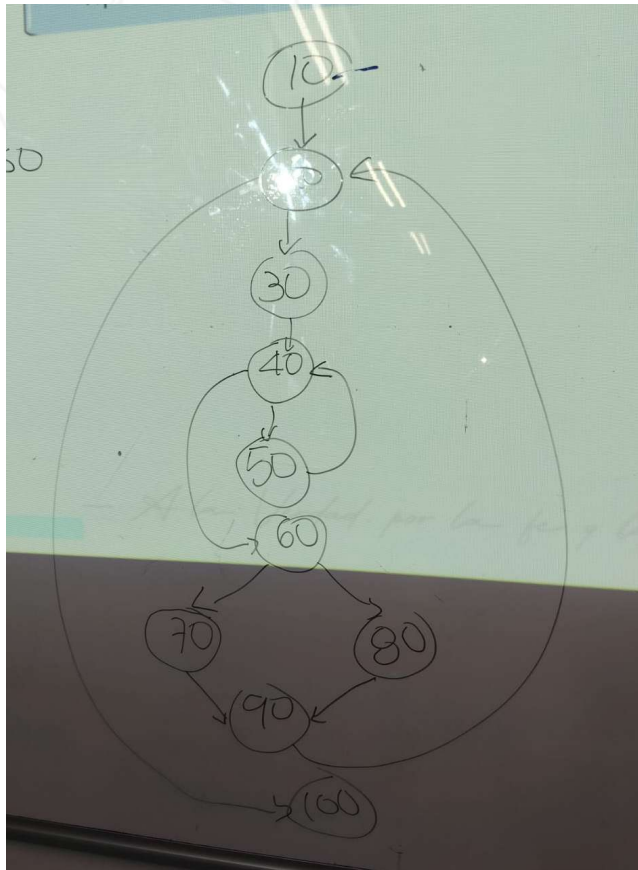
$$v(G) = rr + 1$$



A la Verdad, por la fe y la ciencia

Pruebas de Caja Blanca. Camino Básico

Se determina un conjunto básico de caminos independientes



A la Verdad, por la fe y la ciencia

Pruebas de Caja Blanca. Camino Básico

Se preparan los casos de prueba que obliguen a la ejecución de cada camino del conjunto básico

Entrada

- Código
- Grafo asociado
- $V(G)$

Proceso

- Se cubrirán todos los caminos básicos.

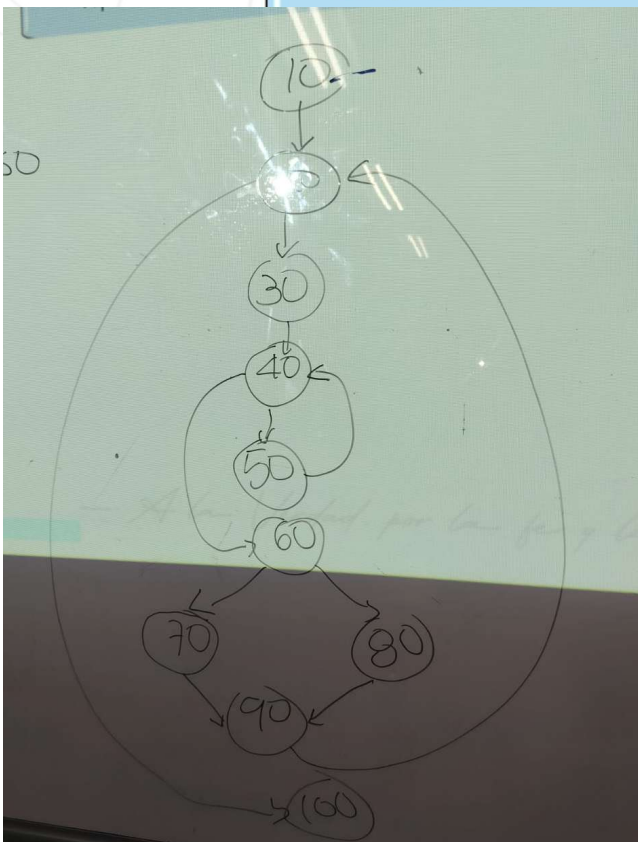
Salida

- Conjunto de casos de prueba, uno por cada camino generado

A la Verdad, por la fe y la ciencia

Pruebas de Caja Blanca. Camino Básico

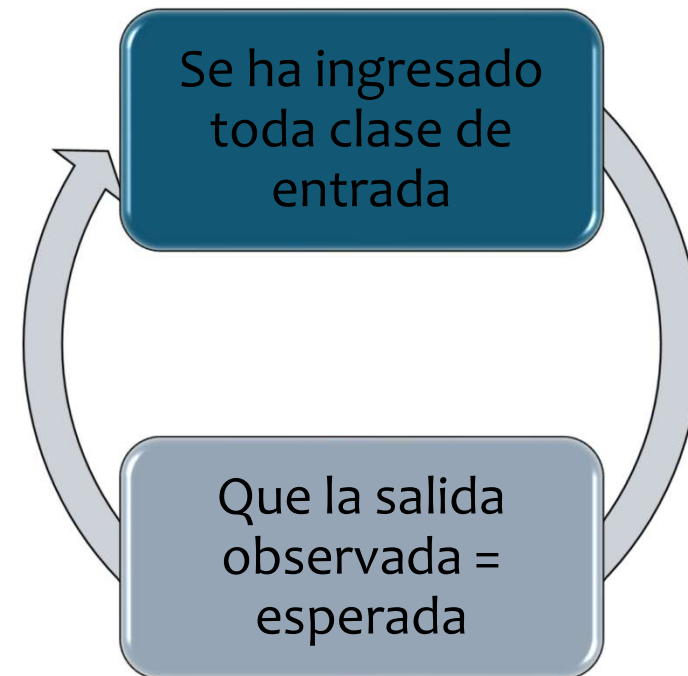
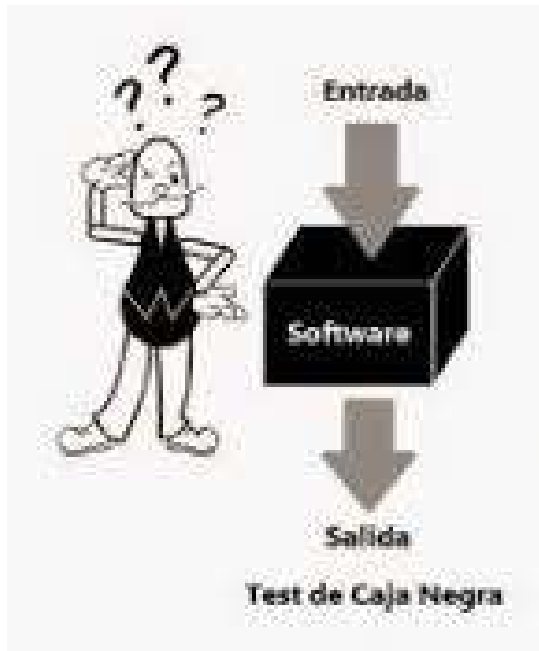
Se preparan los casos de prueba que obliguen a la ejecución de cada camino del conjunto básico



A la Verdad, por la fe y la ciencia

Prueba de Caja Negra

Orientadas a los requisitos funcionales



A la Verdad, por la fe y la ciencia

Prueba de Caja Negra

Particionamiento Equivalente

- ❑ El objetivo es dividir el conjunto completo de posibles entradas en grupos (o clases) de equivalencia, de modo que la suposición es que cualquier valor de entrada dentro de una clase específica debería dar como resultado el mismo comportamiento del sistema.
- ❑ Cómo Funciona el Particionamiento Equivalente

Identificación de Clases de Equivalencia:

Se dividen todas las posibles entradas del sistema en clases de equivalencia.

Definición de Clases Válidas e Inválidas:

Clases Válidas: Estas son clases de entradas que son aceptables.

Clases Inválidas: Estas son clases de entradas que no son válidas o no deberían ser aceptadas por el software.

Selección de Casos de Prueba:

Se selecciona al menos un caso de prueba de cada clase de equivalencia.

dad. por la fe y la ciencia

Prueba de Caja Negra

Particionamiento Equivalente

Identificación de Clases de Equivalencia:

Considera un campo de entrada para la edad de un usuario en un formulario de registro, donde la edad debe estar entre 18 y 65 años inclusive

Definición de Clases Válidas e Inválidas:

Clases Válidas:

- Edad dentro del rango permitido (18-65). Ejemplo de valores: 18, 30, 65.

Clases Inválidas:

- Edad menor que el rango permitido (<18). Ejemplo de valores: 10, 17.
- Edad mayor que el rango permitido (>65). Ejemplo de valores: 66, 70.

Selección de Casos de Prueba:

Un valor de entrada de 30 (válido dentro del rango).

Un valor de entrada de 17 (inválido, menor que el rango).

Un valor de entrada de 66 (inválido, mayor que el rango).

Prueba de Caja Negra

Particionamiento Equivalente

Configuración de Seguimiento

+ Adicionar

PORCENTAJE	DESCRIPCIÓN
Porcentaje:	
Descripción:	Ingrese una descripción del seguimiento
Grabar Cancelar	

Condición de Entrada	Clases Válidas	Clases Inválidas

A la Verdad por la fe y la ciencia

Prueba de Caja Negra

Particionamiento Equivalente

- Diseño de casos de Prueba



A la Verdad, por la fe y la ciencia

Prueba de Caja Negra

Particionamiento Equivalente

- Ejercicio

< Registro

Registrate

✉ - Correo electrónico:

🔒 - Contraseña:

👤 - Nombre:

🧑 - Altura: Altura en Centímetros

💓 - Peso: Peso en Kilogramos

♂ - Género:

Por la Verdad, por la fe y la ciencia

Referencias:

- ❑ Kaner, C., Bach, J., & Pettichord, B. (2002). Lessons Learned in Software Testing. Wiley. ISBN: 978-0471081128.
- ❑ Myers, G. J., Sandler, C., & Badgett, T. (2011). The Art of Software Testing (3rd ed.). Wiley. ISBN: 978-1118031964.
- ❑ Jorgensen, P. C. (2013). Software Testing: A Craftsman's Approach (4th ed.). Auerbach Publications. ISBN: 978-1466560680.
- ❑ Crispin, L., & Gregory, J. (2009). Agile Testing: A Practical Guide for Testers and Agile Teams. Addison-Wesley. ISBN: 978-0321534460.
- ❑ Graham, D., Black, R., & van Veenendaal, E. (2012). Foundations of Software Testing: ISTQB Certification (3rd ed.). Cengage Learning. ISBN: 978-1408044056.

A la Verdad, por la fe y la ciencia

Referencias:

- ❑ Whittaker, J. A. (2009). Exploratory Software Testing. Addison-Wesley. ISBN: 978-0321636416.
- ❑ Kinsbruner, E., Craske, A., & Feloney, S. (2019). Continuous Testing for DevOps Professionals. Apress. ISBN: 978-1484253505.
- ❑ Weinberg, G. M. (2008). Perfect Software and Other Illusions About Testing. Dorset House. ISBN: 978-0932633699.
- ❑ Whittaker, J. A. (2002). How to Break Software: A Practical Guide to Testing. Addison-Wesley. ISBN: 978-0201796193.
- ❑ Black, R. (2012). Advanced Software Testing - Vol. 1: Guide to the ISTQB Advanced Certification as an Advanced Test Analyst (2nd ed.). Rocky Nook. ISBN: 978-1937538497.

A la Verdad, por la fe y la ciencia



¡Gracias!